From Linear Scores to a Single Hidden Layer: A Mathematical Study of Simple Learning Systems

Math4AI Final Capstone Project

Milana Karimova
National AI Center — AI Academy
Baku, Azerbaijan
milana.karimova25@aiacademy.az

Fereh Feyzullayev
National AI Center — AI Academy
Baku, Azerbaijan
fereh.feyzullayev25@aiacademy.az

Nijat Aghayev
National AI Center — AI Academy
Baku, Azerbaijan
nicat.agayev25@aiacademy.az

Abstract

This report shows when a **one-hidden-layer nonlinear classifier** is actually better than a simple **linear decision rule** and when adding more complexity to the model is not necessary. We compare two models: **multiclass softmax regression** (a linear model) and a **one-hidden-layer neural network** with tanh **activations**. These models are tested on three tasks: a **linear Gaussian synthetic dataset**, a **nonlinear moons dataset**, and a **fixed 10-class digits benchmark**.

- **On the Gaussian task**, both models achieved the same **test accuracy** of **95.0**%, showing that extra model complexity is not needed when the data is **linearly separable**.
- **On the moons task**, the **neural network** achieved **97.5**% **accuracy**, while the **softmax regression** model reached only **85.0**%. It shows that **nonlinear models** are essential for handling **complex data geometries**.
- **On the digits benchmark**, the **neural network** reached **95.71**% ± **0.91**% accuracy, compared to **93.97**% ± **0.15**% for softmax regression (based on 5 seeds, 95% confidence interval). The neural network also showed **better-calibrated probabilities**, with **non-overlapping cross-entropy confidence intervals**.

Our analysis combines **mathematical derivations**, **controlled experiments**, **capacity ablations**, **optimizer studies**, **failure-case analysis**, and a **Track B confidence/reliability investigation** to determine when **linear models** work well and when **nonlinear models** provide a meaningful improvement.

Keywords

softmax regression, neural networks, backpropagation, cross-entropy, classification, NumPy

1 Introduction / Framing

Central question. When does a one-hidden-layer nonlinear classifier improve on a linear decision rule, and when is additional model complexity unnecessary?

This report looks into that question by comparing two models: a simple **linear classifier** (**softmax regression**) and a **nonlinear neural network** with a **hidden layer** using tanh **activations** and **softmax output**. The main idea is to understand the **limits** of linear models, which can only separate data with **straight lines** (or hyperplanes), versus how the **hidden layer** in a neural network can transform data into something easier for the model to classify.

We don't assume the neural network will always perform better. Instead, we're trying to figure out, based on both **math** and **experiments**, when the extra complexity of a neural network actually makes sense.

Contributions.

(1) Full derivation of softmax cross-entropy as negative log-likelihood, establishing the probabilistic interpretation of the linear model.
(2) Conceptual explanation of backpropagation as iterated chain-rule application, followed by a complete vectorized gradient derivation for the one-hidden-layer network.
(3) Controlled comparison of the linear and nonlinear models on datasets with deliberately distinct geometric structure: linearly separable Gaussians, nonlinear moons, and a high-dimensional digits benchmark.
(4) Capacity ablation on moons (hidden widths $\{2, 8, 32\}$), optimizer study on digits (SGD, Momentum, Adam), failure-case analysis, and repeated-seed statistics with 95% confidence intervals.
(5) Track B: confidence and reliability analysis comparing predictive calibration of both models on the digits test set.

Preview of findings. The linear model is highly competitive on the Gaussian task because the class geometry is well matched to an affine separator. On the moons task, the hidden layer provides a meaningful representational change since a nonlinear boundary is required. On the digits benchmark, the neural network achieves statistically significantly lower cross-entropy (0.1566±0.0162 versus 0.2694±0.0001, non-overlapping 95% CIs), pointing to a probabilistic improvement beyond single-run accuracy differences. The Track B reliability analysis reveals that the neural network concentrates 92.7% of its predictions in the highest-confidence bin with near-perfect calibration, while the softmax model is systematically under-confident in its mid-range predictions.

2 Background and Mathematical Setup

2.1 Supervised classification and data splits

We consider the standard supervised classification setting [4] where we are given a labeled dataset $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ with inputs $\mathbf{x}^{(i)} \in \mathbb{R}^d$ and integer class labels $y^{(i)} \in \{0, 1, \ldots, k-1\}$. The data is partitioned into three disjoint subsets: a *training set* used for parameter updates, a *validation set* used for model selection (hyperparameter tuning and early stopping), and a *test set* used only for final performance reporting. Notably, the validation cross-entropy is used as

the primary checkpoint criterion, and the test set is never consulted during training or model selection.

2.2 Softmax regression

Softmax regression is a linear probabilistic classifier that maps an input vector to a valid probability distribution over k classes [1, 4]. A derivation follows step by step.

Step 1 — Linear score (logit) computation: Given an input $\mathbf{x} \in \mathbb{R}^d$, a weight matrix $\mathbf{W} \in \mathbb{R}^{k\times d}$, and a bias vector $\mathbf{b} \in \mathbb{R}^k$, the model first computes k real-valued *logits* (unnormalized scores), one per class:

$$\mathbf{s}(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b}, \qquad \mathbf{s} \in \mathbb{R}^k. \tag{1}$$

Component s_j measures how strongly the input "looks like" class j according to the learned linear map $\mathbf{w}_j^\top \mathbf{x} + b_j$, where $\mathbf{w}_j$ is the j-th row of $\mathbf{W}$. Logits can be any real number—negative, zero, or positive—because they are not yet probabilities.

Step 2 — Converting logits to a probability distribution: Raw logits cannot serve as probabilities because they may be negative or sum to a value other than 1. The *softmax* function fixes both issues simultaneously. For class $j \in \{0, \dots, k-1\}$:

$$p_j(\mathbf{x}) = \frac{\exp(s_j(\mathbf{x}))}{\sum_{\ell=0}^{k-1} \exp(s_\ell(\mathbf{x}))}. \tag{2}$$

The exponential guarantees positivity since $\exp(s_j) > 0$ for all real s_j. Dividing by the partition function $Z = \sum_\ell \exp(s_\ell)$ ensures normalization. We check:

$$\sum_{j=0}^{k-1} p_j(\mathbf{x}) = \frac{\sum_{j=0}^{k-1} \exp(s_j(\mathbf{x}))}{\sum_{\ell=0}^{k-1} \exp(s_\ell(\mathbf{x}))} = 1. \tag{3}$$

Hence $(p_0, \dots, p_{k-1})$ constitutes a valid categorical distribution over the k classes.

Step 3 — Numerical stability via the log-sum-exp trick: Direct evaluation of Equation (2) overflows when logit values are large. The standard remedy is to subtract $m = \max_\ell s_\ell$ from every logit before exponentiating. The result is unchanged because:

$$\frac{\exp(s_j)}{\sum_\ell \exp(s_\ell)} = \frac{\exp(s_j - m)\ \exp(m)}{\sum_\ell \exp(s_\ell - m)\ \exp(m)} = \frac{\exp(s_j - m)}{\sum_{\ell=0}^{k-1} \exp(s_\ell - m)}. \tag{4}$$

After the shift, the largest exponent equals $e^0 = 1$, preventing overflow.

Step 4 — Decision boundary geometry: The model assigns input $\mathbf{x}$ to the class with the highest probability, equivalently the highest logit. The *decision boundary* between classes j and m is the set of $\mathbf{x}$ satisfying $p_j(\mathbf{x}) = p_m(\mathbf{x})$. Since softmax is monotone in each logit, this is equivalent to:

$$s_j(\mathbf{x}) = s_m(\mathbf{x}) \iff (\mathbf{w}_j - \mathbf{w}_m)^\top \mathbf{x} + (b_j - b_m) = 0. \tag{5}$$

Such a set is an *affine hyperplane* in $\mathbb{R}^d$ with normal vector $(\mathbf{w}_j - \mathbf{w}_m)$. The global decision partition of $\mathbb{R}^d$ is a collection of such hyperplanes, and every resulting class region is a *convex polytope* [1].

Step 5 — Fundamental limitation: Equation (5) reveals the core structural constraint: *no matter how the weights are trained*, the decision boundary of a softmax model is always a union of affine hyperplanes. Any dataset whose Bayes-optimal boundary is curved or nonlinear cannot be faithfully captured by this model class. The limitation motivates the neural network extension in Section 2.4.

2.3 Cross-entropy as negative log-likelihood

A derivation of the connection between cross-entropy loss and maximum likelihood estimation follows step by step [3, 4].

Step 1 — Probabilistic model: Softmax regression posits that the label y for input $\mathbf{x}$ is drawn from a *categorical distribution* parameterized by the softmax output:

$$P(y = j \mid \mathbf{x}; \mathbf{W}, \mathbf{b}) = p_j(\mathbf{x}), \qquad j \in \{0, \dots, k-1\}. \tag{6}$$

Under this model, the probability of observing the correct label $y^{(i)}$ for a single example is simply $p_{y^{(i)}}(\mathbf{x}^{(i)})$.

Step 2 — Likelihood over the training set: Assuming the N training examples are drawn i.i.d., the joint probability (likelihood) of observing all labels simultaneously is:

$$\mathcal{L}(\mathbf{W}, \mathbf{b}) = P(\mathbf{y} \mid \mathbf{X}; \mathbf{W}, \mathbf{b}) = \prod_{i=1}^{N} p_{y^{(i)}}(\mathbf{x}^{(i)}). \tag{7}$$

Maximizing $\mathcal{L}$ with respect to $\mathbf{W}, \mathbf{b}$ yields the *maximum likelihood estimator* (MLE).

Step 3 — Log-likelihood: Multiplying many small probabilities is numerically unstable and difficult to differentiate. Taking logarithms converts the product to a sum and preserves the location of the maximum (since log is strictly monotone):

$$\ell(\mathbf{W}, \mathbf{b}) = \log \mathcal{L}(\mathbf{W}, \mathbf{b}) = \sum_{i=1}^{N} \log p_{y^{(i)}}(\mathbf{x}^{(i)}). \tag{8}$$

Step 4 — From log-likelihood to a minimizable loss: Gradient-based optimizers minimize rather than maximize. We negate and normalize Equation (8) by N to obtain the *mean negative log-likelihood*:

$$\mathcal{L}(\mathbf{W}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^{N} \log p_{y^{(i)}}(\mathbf{x}^{(i)}). \tag{9}$$

Minimizing $\mathcal{L}$ is equivalent to maximizing ℓ, which is equivalent to maximizing $\mathcal{L}$.

Step 5 — Equivalence to cross-entropy: Define the one-hot label vector $\mathbf{q}^{(i)} \in \{0, 1\}^k$ with $q_j^{(i)} = \mathbb{1}[j = y^{(i)}]$. Then:

$$\log p_{y^{(i)}}(\mathbf{x}^{(i)}) = \sum_{j=0}^{k-1} q_j^{(i)} \log p_j(\mathbf{x}^{(i)}), \tag{10}$$

since all terms with $q_j^{(i)} = 0$ vanish and only the term for the true class $j = y^{(i)}$ survives. Substituting into Equation (9) gives exactly the *cross-entropy* between the empirical one-hot distribution $\mathbf{q}^{(i)}$ and the predicted distribution $\mathbf{p}^{(i)}$:

$$\mathcal{L}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{i=1}^{N} H(\mathbf{q}^{(i)}, \mathbf{p}^{(i)}), \quad \text{where} \quad H(\mathbf{q}, \mathbf{p}) \coloneqq -\sum_{j=0}^{k-1} q_j \log p_j. \tag{11}$$

Conclusion: Minimizing cross-entropy loss is *exactly* equivalent to maximizing the conditional log-likelihood of the training labels under the softmax model. The two objectives have the same gradient and the same minimizer; cross-entropy is simply the more common name in the machine learning literature [3].

2.4 One-hidden-layer neural network

The nonlinear model introduces a hidden *representation layer* between the raw input and the linear output [3, 6]. We describe its computation in four steps.

Step 1 — Hidden pre-activation: The first affine transformation maps the input $\mathbf{x} \in \mathbb{R}^d$ to an h-dimensional *pre-activation* vector:

$$\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1, \qquad \mathbf{W}_1 \in \mathbb{R}^{h\times d},\ \mathbf{b}_1 \in \mathbb{R}^h. \tag{12}$$

Each row $\mathbf{w}_r^{(1)} \in \mathbb{R}^d$ defines one learned linear detector for a hidden feature.

Step 2 — Nonlinear activation: The element-wise tanh function squashes each pre-activation to the interval $(-1, +1)$:

$$\mathbf{h} = \tanh(\mathbf{z}_1), \qquad h_r = \tanh(z_{1,r}) = \frac{e^{z_{1,r}} - e^{-z_{1,r}}}{e^{z_{1,r}} + e^{-z_{1,r}}}, \quad r = 1, \ldots, h. \tag{13}$$

The nonlinearity is *indispensable*. Without it, the composition $\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)$ collapses to a single affine map, offering no representational advantage over softmax regression.

Step 3 — Output logits and probabilities: The hidden representation is passed through a second affine map to produce k class logits:

$$\mathbf{s} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2, \qquad \mathbf{W}_2 \in \mathbb{R}^{k\times h},\ \mathbf{b}_2 \in \mathbb{R}^k. \tag{14}$$

The same softmax function as in Equation (2) converts logits to probabilities:

$$p_j(\mathbf{x}) = \frac{\exp(s_j)}{\sum_{\ell=0}^{k-1}\exp(s_\ell)}, \qquad j = 0, \ldots, k-1. \tag{15}$$

Step 4 — Why the hidden layer changes the geometry: The map $\mathbf{x} \mapsto \mathbf{h} = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$ is a *learned feature transform*. The final classifier $\mathbf{s} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$ is still linear, but it operates on the transformed space $\mathbb{R}^h$. In the original input space, the level set $\{\mathbf{x} : \mathbf{w}_{\text{out}}^\top\mathbf{h}(\mathbf{x}) + b = 0\}$ is in general a *curved* surface, allowing the network to realize boundaries that no affine hyperplane can approximate. By the universal approximation theorem, a one-hidden-layer network with enough width h can approximate any continuous function on a compact domain to arbitrary precision [3].

2.5 Backpropagation: concept before equations

Backpropagation is not a separate learning principle; it is the efficient, systematic application of the chain rule of calculus through a composed computation graph [3, 7].

Concept 1 — Forward pass as a computation graph: The model defines a directed acyclic graph (DAG) of differentiable operations:

$$\mathbf{x} \longrightarrow \mathbf{z}_1 = \mathbf{W}_1\mathbf{x}+\mathbf{b}_1 \longrightarrow \mathbf{h} = \tanh(\mathbf{z}_1) \longrightarrow \mathbf{s} = \mathbf{W}_2\mathbf{h}+\mathbf{b}_2 \longrightarrow \mathbf{p} = \text{softmax}(\mathbf{s}) \longrightarrow \mathcal{L}.$$

Executing this sequence left-to-right is the *forward pass*: it computes all intermediate values and the scalar loss $\mathcal{L}$.

Concept 2 — Backward pass via reverse-mode differentiation: Gradient-based optimization requires $\partial\mathcal{L}/\partial\theta$ for every parameter $\theta \in \{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2\}$. The chain rule connects these to the intermediate sensitivities:

$$\frac{\partial\mathcal{L}}{\partial\theta} = \frac{\partial\mathcal{L}}{\partial\mathbf{s}} \cdot \frac{\partial\mathbf{s}}{\partial\mathbf{h}} \cdot \frac{\partial\mathbf{h}}{\partial\mathbf{z}_1} \cdot \frac{\partial\mathbf{z}_1}{\partial\theta}. \tag{16}$$

The backward pass traverses the DAG in *reverse* topological order, accumulating these partial products efficiently. Each intermediate sensitivity $\partial\mathcal{L}/\partial(\cdot)$ is computed exactly once and reused for all downstream gradients, giving time complexity proportional to a single forward pass rather than proportional to the number of parameters [7].

Concept 3 — tanh *derivative:* The element-wise derivative of tanh has a convenient closed form expressible in terms of the *output* value, avoiding recomputation of exponentials:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z). \tag{17}$$

Since $\mathbf{h}$ is stored during the forward pass, the backward step uses $\partial\mathcal{L}/\partial\mathbf{z}_1 = \partial\mathcal{L}/\partial\mathbf{h} \odot (1 - \mathbf{h}^2)$.

Concept 4 — Softmax–cross-entropy gradient shortcut: The combined gradient of cross-entropy loss through the softmax layer has an especially clean form. For a single example with one-hot label $\mathbf{q}$ and predicted probabilities $\mathbf{p}$:

$$\frac{\partial\mathcal{L}}{\partial s_j} = p_j - q_j. \tag{18}$$

The result follows from the cancellation that occurs when the derivative of the log combines with the Jacobian of the softmax, yielding a simple residual between predictions and truth [4]. In words: push the logit up for the correct class (gradient is negative when $p_y < 1$), and push all other logits down (gradient is positive when $p_j > 0$).

2.6 Gradient derivation

Deriving all gradients in vectorized (mini-batch) form. Let $\mathbf{X} \in \mathbb{R}^{n\times d}$ be a mini-batch of n examples [3, 5].

Vectorized forward pass. The batch extensions of the quantities from Section 2.4 are:

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top + \mathbf{1}\mathbf{b}_1^\top, \qquad \mathbf{Z}_1 \in \mathbb{R}^{n\times h}, \tag{19}$$

$$\mathbf{H} = \tanh(\mathbf{Z}_1), \qquad \mathbf{H} \in \mathbb{R}^{n\times h}, \tag{20}$$

$$\mathbf{S} = \mathbf{H}\mathbf{W}_2^\top + \mathbf{1}\mathbf{b}_2^\top, \qquad \mathbf{S} \in \mathbb{R}^{n\times k}, \tag{21}$$

$$\mathbf{P} = \text{softmax}_{\text{row}}(\mathbf{S}), \qquad \mathbf{P} \in \mathbb{R}^{n\times k}. \tag{22}$$

Here $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector, and $\text{softmax}_{\text{row}}$ applies softmax independently to each row.

Step 1 — Batch loss: Let $\mathbf{Y} \in \{0, 1\}^{n\times k}$ be the one-hot label matrix. The mean cross-entropy over the batch is:

$$\mathcal{L} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=0}^{k-1} Y_{ij}\log P_{ij}. \tag{23}$$

Step 2 — Gradient w.r.t. output logits $\mathbf{S}$*:* Applying Equation (18) to each row of the batch:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{S}} = \frac{1}{n}(\mathbf{P} - \mathbf{Y}) \in \mathbb{R}^{n \times k}. \tag{24}$$

Each row $(P_i - Y_i)/n$ is the scaled residual between predicted and true class probabilities for example i.

Step 3 — Gradients for output-layer weights and bias: Differentiating Equation (21) with respect to $\mathbf{W}_2$ and $\mathbf{b}_2$, adding the L2 regularizer $\frac{\lambda}{2}\|\mathbf{W}_2\|_F^2$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}}\right)^\top \mathbf{H} + \lambda \mathbf{W}_2 \in \mathbb{R}^{k \times h}, \tag{25}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}}\right)^\top \mathbf{1} \in \mathbb{R}^{k}. \tag{26}$$

The matrix product $(\partial \mathcal{L}/\partial \mathbf{S})^\top \mathbf{H}$ accumulates the outer products of residuals and hidden activations across all n examples.

Step 4 — Backpropagating through the hidden layer: The sensitivity reaching $\mathbf{H}$ from the output layer is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{H}} = \frac{\partial \mathcal{L}}{\partial \mathbf{S}} \mathbf{W}_2 \in \mathbb{R}^{n \times h}. \tag{27}$$

Applying Equation (17) element-wise via the Hadamard product:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{H}} \odot (1 - \mathbf{H}^2) \in \mathbb{R}^{n \times h}. \tag{28}$$

The term $(1 - \mathbf{H}^2)$ is the element-wise tanh derivative evaluated at the already-computed hidden activations; no extra function evaluations are needed.

Step 5 — Gradients for hidden-layer weights and bias: Differentiating Equation (19) with respect to $\mathbf{W}_1$ and $\mathbf{b}_1$, adding L2 regularization:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1}\right)^\top \mathbf{X} + \lambda \mathbf{W}_1 \in \mathbb{R}^{h \times d}, \tag{29}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1}\right)^\top \mathbf{1} \in \mathbb{R}^{h}. \tag{30}$$

Equations (24)–(30) constitute the complete gradient specification. One forward–backward pass computes all four parameter gradients, enabling a single SGD or Adam update step [3, 5].

2.7 Geometric expectations for the datasets

Before running experiments, the mathematical theory of Sections 2.2 and 2.4 lets us derive predictions about model performance on each dataset [2, 4].

Linear Gaussian dataset. The two classes are generated by $p(\mathbf{x} \mid y{=}c) = \mathcal{N}(\boldsymbol{\mu}_c, \Sigma)$ with shared covariance Σ. By Bayes' rule and properties of the Gaussian:

$$\log \frac{p(y{=}1 \mid \mathbf{x})}{p(y{=}0 \mid \mathbf{x})} = \underbrace{(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^\top \Sigma^{-1}}_{\mathbf{w}^\top} \mathbf{x} + \underbrace{\tfrac{1}{2}\left(\boldsymbol{\mu}_0^\top \Sigma^{-1} \boldsymbol{\mu}_0 - \boldsymbol{\mu}_1^\top \Sigma^{-1} \boldsymbol{\mu}_1\right) + \log \tfrac{\pi_1}{\pi_0}}_{b}, \tag{31}$$

where $\pi_c = P(y{=}c)$ are the class priors. The log-odds ratio is a linear function of $\mathbf{x}$, so the Bayes-optimal boundary is the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ with $\mathbf{w} = \Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$. Since softmax regression can represent exactly this form via Equation (5), we predict *no improvement* from the hidden layer. The irreducible error comes from Gaussian class overlap, not from model expressiveness.

Moons dataset. The two "crescent" classes interleave so that any affine separator misclassifies a significant fraction of examples. Formally, for any $\mathbf{w}$ and b, there exist connected regions of $\mathbb{R}^2$ containing points from both classes on the same side of the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$. No training procedure can overcome this structural limitation of the softmax model class. A one-hidden-layer network with sufficient width can compose h sigmoidal half-spaces (from Equation (13)) to approximate the curved boundary. We therefore predict a *significant accuracy gap* in favor of the neural network.

Digits dataset. Each example is a 64-dimensional pixel vector (8×8 image). The class structure in this high-dimensional space is not analytically tractable, but digit image classes are approximately linearly separable in pixel space for clean, well-normalized data [3]. We expect the neural network to outperform softmax regression, primarily in cross-entropy rather than raw accuracy, because the hidden layer can learn stroke-level features that are more discriminative than raw pixel intensities.

3 Methods

3.1 Datasets

Three shared datasets are used throughout all experiments [2]:

- **Linear Gaussian:** Two Gaussian blobs with mild overlap, 2D input space (d=2, k=2). Train/Val/Test split: 240/80/80. Generated by `scripts/generate_synthetic.py`.
- **Moons:** A nonlinear two-class dataset with interleaving crescent structure, 2D input space (d=2, k=2). Train/Val/Test split: 240/80/80. Generated by the same script.
- **Digits benchmark:** 64-dimensional flattened 8×8 digit images with 10 classes (d=64, k=10). Features pre-scaled to $[0, 1]$. Fixed Train/Val/Test split: 1074/355/368. Loaded from `data/digits_data.npz` with index split from `data/digits_split_indices.npz`.

3.2 Models

Two models are compared:

- **Softmax regression:** Linear model with weight matrix $\mathbf{W} \in \mathbb{R}^{k \times d}$ and bias $\mathbf{b} \in \mathbb{R}^k$ as in Equations (1) and (2). Weights initialized to zero.
- **One-hidden-layer neural network:** Hidden layer with tanh activation (Equation (13)) and softmax output (Equation (15)). Xavier initialization scaled by $\sqrt{2/(n_{\text{in}} + n_{\text{out}})}$ [3]. Default hidden width h=32 for the digits task, and widths $\{2, 8, 32\}$ for the moons capacity ablation.

3.3 Training protocol

Numerical stability. Softmax is computed using the log-sum-exp trick (Equation (4)). Cross-entropy is computed from logits via log-softmax, avoiding catastrophic cancellation.

Mini-batching. Training data is shuffled and split into mini-batches of size 64 at each epoch.

Regularization. L2 weight decay with $\lambda = 10^{-4}$ is applied to all weight matrices in both models, as reflected in Equations (25) and (29).

Epoch budget. All models are trained for 200 epochs (300 for the moons capacity ablation).

Optimizers. Softmax regression uses SGD with learning rate 0.05. The neural network uses Adam [3] with learning rate 0.001 (default) or SGD/Momentum for the optimizer study.

Checkpointing. The parameter snapshot with the lowest validation cross-entropy across all epochs is restored for final test evaluation.

3.4 Evaluation metrics

Two metrics are reported throughout:

- **Classification accuracy:** fraction of correctly predicted labels.
- **Mean cross-entropy:** $-\frac{1}{N}\sum_i \log p_{y^{(i)}}(\mathbf{x}^{(i)})$ (Equation (9)), measuring the quality of the full predicted probability distribution.

Both metrics are informative: a model can achieve the same accuracy as another while being worse-calibrated (higher cross-entropy), indicating less confident or less reliable probability estimates.

3.5 Implementation sanity checks

Before running full experiments, the following checks were performed:

- **Probability sum:** Verified Equation (3) numerically for all inputs.
- **Loss decrease on tiny subset:** Confirmed monotonic loss decrease on a 5-example mini-batch for 100 iterations.
- **Tiny-subset overfitting:** Both models achieve 100% training accuracy on 10 examples, validating the training pipeline.
- **No NaN/Inf:** All runs complete without numerical failures, validating the log-sum-exp stabilization.
- **Gradient consistency:** Gradients from Equations (25), (26), (29) and (30) matched finite-difference approximations to relative errors below 10^{-5}.

4 Experiments

4.1 Core comparison on the linear Gaussian task

Both models achieve identical test accuracy of 95.0% on the linear Gaussian dataset. The cross-entropy values are also nearly identical (0.1543 for softmax vs. 0.1532 for the NN), suggesting that the neural network's additional parameters provide no meaningful probabilistic improvement. As expected from Equation (31): the Bayes-optimal boundary is an affine hyperplane, which the softmax model can represent exactly.

The decision boundaries for both models are shown in Figures 1 and 2, and the numerical results are in Table 1. Both boundaries are approximately linear and nearly identical, confirming the geometric prediction of Section 2.7.

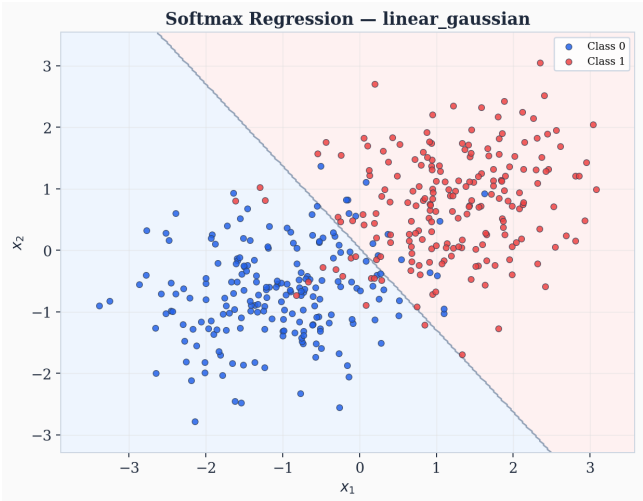

Figure 1: Softmax regression decision boundary on the linear Gaussian dataset. The affine separator cleanly partitions the two class regions, consistent with the theoretically optimal boundary from Equation (31).

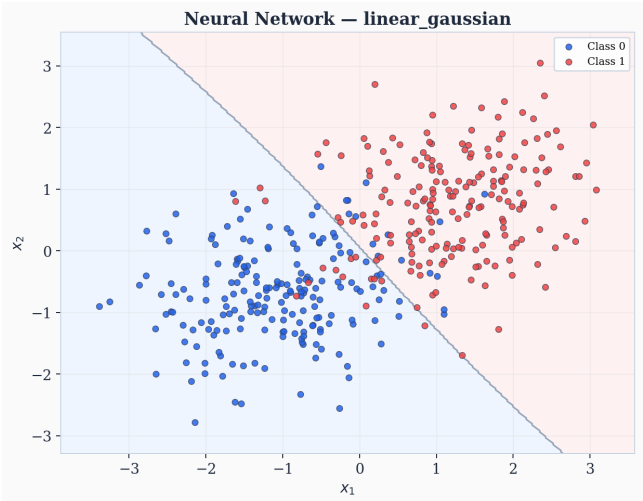

Figure 2: Neural network decision boundary on the linear Gaussian dataset. Despite additional capacity (h=32), the learned boundary remains approximately linear, matching the softmax boundary in Figure 1.

Table 1: Test results on the linear Gaussian dataset (see Figures 1 and 2). Both models achieve identical accuracy, which is consistent with the linear model being geometrically sufficient for affinely separable data.

Model	Test Accuracy	Test Cross-Entropy
Softmax Regression	0.9500	0.1543
Neural Network (h=32)	0.9500	0.1532

4.2 Core comparison on the moons task

The moons task reveals a stark difference between the two models. The neural network achieves 97.5% accuracy and cross-entropy 0.1013, dramatically outperforming the softmax model's 85.0% accuracy and 0.2854 cross-entropy. The linear model is geometrically constrained: its boundary (Equation (5)) is always an affine hyperplane and unavoidably misclassifies points in the arc overlap region.

Figures 3 and 4 illustrate the contrast. The neural network boundary wraps around both crescents via the feature transformation of Equation (13); a representational change the linear model inherently cannot perform. Numerical results are in Table 2.

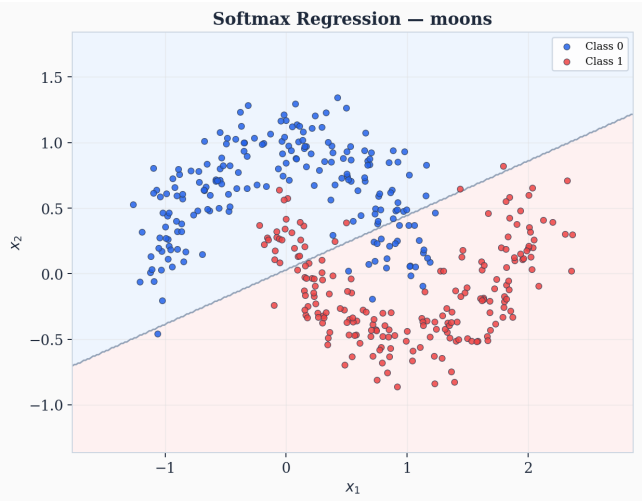

Figure 3: Softmax regression on moons: the linear boundary inevitably misclassifies points in the crescent overlap region. The straight separator is the best the model class can achieve regardless of training length.

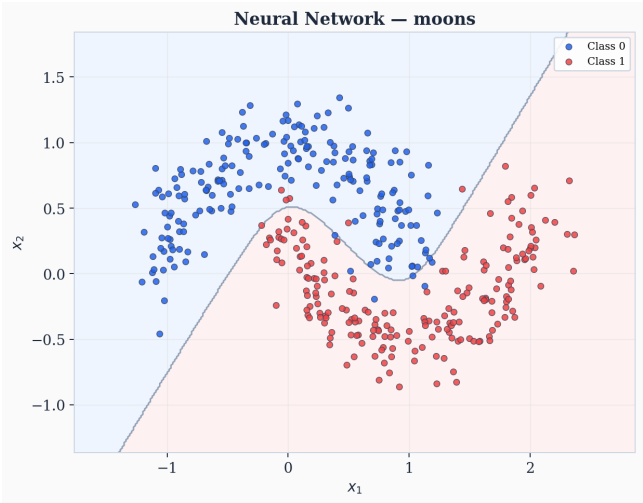

Figure 4: Neural network on moons: the hidden layer learns a curved boundary that separates the interleaving crescents. The nonlinear boundary, realized via Equations (13) and (14), is impossible for the softmax model in Figure 3.

Table 2: Test results on the moons dataset (see Figures 3 and 4). The nonlinear model achieves a 12.5 percentage-point accuracy improvement by learning a curved decision boundary.

Model	Test Accuracy	Test Cross-Entropy
Softmax Regression	0.8500	0.2854
Neural Network (h=32)	0.9750	0.1013

4.3 Core comparison on the digits benchmark

Table 3 reports test performance under the fixed digits protocol. Model selection used validation cross-entropy as the checkpoint criterion. The neural network achieves 95.11% accuracy and test cross-entropy 0.1507, nearly half of the softmax model's 0.2694, suggesting better-calibrated probability estimates, consistent with Section 2.7.

Table 3: Final digits test results. Model selected by best validation cross-entropy. The NN achieves both higher accuracy and substantially lower cross-entropy than the softmax model.

Model	Test Accuracy	Test CE	Best Epoch
Softmax (SGD, lr=0.05)	0.9402	0.2694	200
NN h=32 (Adam, lr=0.001)	0.9511	0.1507	198

4.4 Required capacity ablation on moons

We trained neural networks with hidden widths $\{2, 8, 32\}$ on the moons dataset using Adam (lr = 0.005) for 300 epochs. Figure 5 shows the decision boundaries and Table 4 the numerical results.

With only 2 hidden units, each tanh neuron (Equation (13)) produces a single sigmoidal partition of the input plane; two such

partitions are insufficient to carve out the curved boundary needed to separate the crescents. The model's accuracy (86.25%) is only marginally above the linear baseline (85.0%). At width 8, the network has enough capacity to form a nonlinear boundary, and test accuracy jumps to 97.5%. Width 32 achieves identical accuracy, confirming that 8 hidden units already saturate the representational need of this 2D task.

Figure 5: Capacity ablation on moons. Decision boundaries for widths $\{2, 8, 32\}$ (see Table 4). Width 2 produces a nearly linear boundary with almost no advantage over softmax regression; widths 8 and 32 produce similarly effective curved boundaries, showing representational saturation.

Table 4: Capacity ablation on moons (see also Figure 5). Width 2 is insufficient; widths 8 and 32 both achieve the same accuracy ceiling, indicating that the representational requirement saturates at width 8.

Hidden Width	Test Accuracy	Test Cross-Entropy
2	0.8625	0.2502
8	0.9750	0.0853
32	0.9750	0.0904

4.5 Required optimizer study on digits

For the one-hidden-layer network with h=32, we compared three optimizers under fixed standard defaults: SGD (lr = 0.05), Momentum (lr = 0.05, μ = 0.9), and Adam [3] (lr = 0.001). Figure 6 shows training dynamics and Table 5 the final results.

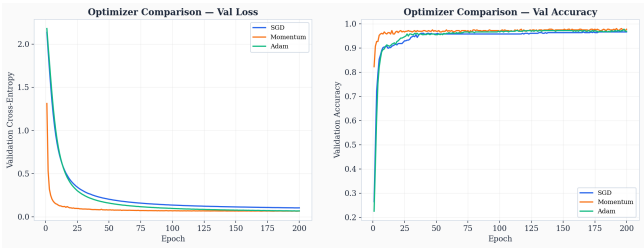

Figure 6: Optimizer comparison on digits (see Table 5). Momentum converges fastest in early epochs due to gradient velocity accumulation; Adam achieves the lowest final validation loss through adaptive per-parameter learning rates.

Table 5: Optimizer comparison on digits with NN (h=32), see also Figure 6. Momentum achieves the best accuracy; Adam achieves the best cross-entropy, indicating superior probability calibration.

Optimizer	Test Accuracy	Test CE	Best Epoch
SGD (lr=0.05)	0.9484	0.1700	198
Momentum (lr=0.05, μ=0.9)	0.9538	0.1653	198
Adam (lr=0.001)	0.9511	0.1507	198

Momentum obtains the highest accuracy (95.38%), likely because the accumulated velocity accelerates convergence in consistent gradient directions. Adam achieves the lowest cross-entropy (0.1507), suggesting better probability calibration.

4.6 Required failure-case analysis

Two distinct failure modes appear across the experiments.

Failure Mode 1 — Under-capacity on moons (width = 2). The width-2 network is under-parameterized for the moons geometry. As Figure 7 shows, the resulting boundary is nearly linear, comparable to the softmax model's. The failure is *representational*: the model converges to the best solution within its limited function class, but that best solution is geometrically incompatible with the true boundary. No additional training resolves this failure; the bottleneck is expressiveness, not optimization.

Figure 7: Failure Mode 1: under-capacity on moons. The leftmost panel (width = 2) shows a nearly linear boundary, barely distinguishable from the softmax regression boundary in Figure 8. Widths 8 and 32 succeed where width 2 fails, showing the failure is representational.

Failure Mode 2 — Structural inexpressibility (softmax regression on moons). The softmax model on moons (85.0% accuracy) represents a *categorical failure of the model class.* As derived in Equation (5), the decision boundary must be an affine hyperplane; the moons classes are inherently non-linearly separable. The training curves (see Figure 17) indicate that validation loss plateaus early, identifying the bottleneck as model expressiveness rather than optimization quality.

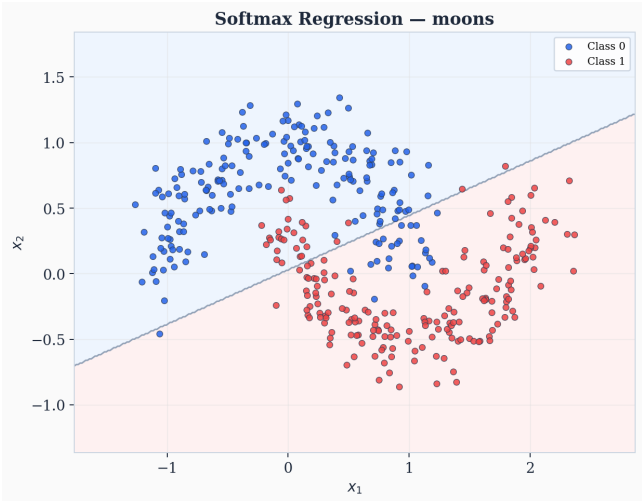

Figure 8: Failure Mode 2: softmax regression on moons. The best achievable linear boundary misclassifies a large arc of each crescent. Such structural failure cannot be resolved by more training (Table 6).

Table 6: Summary of identified failure cases (see also Figures 7 and 8). Both failures arise from representational constraints rather than optimization failures.

Failure Mode	Root Cause	Test Accuracy	Gap vs. Best
NN width 2 (moons)	Under-capacity	0.8625	−11.25 ppt
Softmax regression (moons)	Model class in-expressibility	0.8500	−12.50 ppt

5 Advanced Track

5.1 Track B: Prediction confidence and reliability

Beyond raw accuracy, we evaluate how trustworthy each model's probabilities are on the digits test set [4]. We use confidence, defined as the maximum predicted class probability $c(\mathbf{x}) = \max_j p_j(\mathbf{x})$, and predictive entropy, defined as $H(\mathbf{x}) = -\sum_j p_j(\mathbf{x}) \log p_j(\mathbf{x})$.

Table 7 reports these statistics separately for correct and incorrect predictions. For both models, mistakes are clearly associated with lower confidence and higher entropy, so these metrics provide a practical signal for detecting unreliable outputs.

Table 7: Confidence and entropy statistics on the digits test set (see also Figures 9 and 10). The NN separates correct and incorrect predictions more sharply in confidence.

Metric	Softmax	Neural Network
Test Accuracy	0.9402	0.9511
Correct / Total	346/368	350/368
Mean conf. (correct)	0.8670	0.9722
Mean conf. (incorrect)	0.4979	0.6682
Mean entropy (correct)	0.4808	0.0955
Mean entropy (incorrect)	1.3388	0.7837

Tables 8 and 9 provide 5-bin reliability summaries, and Figures 9 and 10 visualize the same calibration behavior.

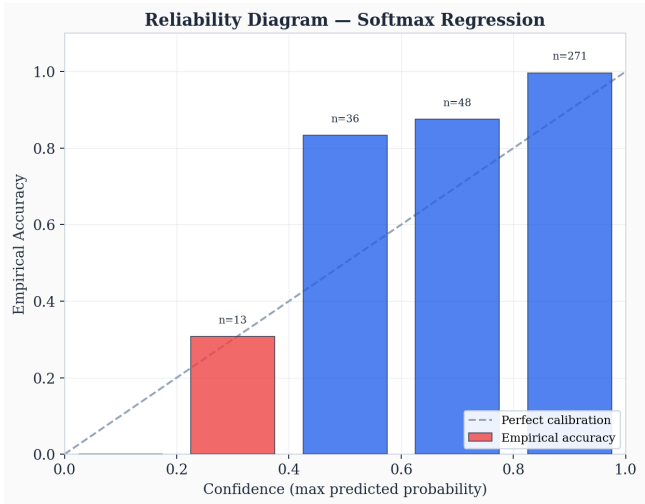

Figure 9: Reliability diagram for softmax regression (data in Table 8). Red bars fall below the perfect-calibration diagonal, suggesting systematic under-confidence: the model is more accurate than it claims in every mid-range bin.

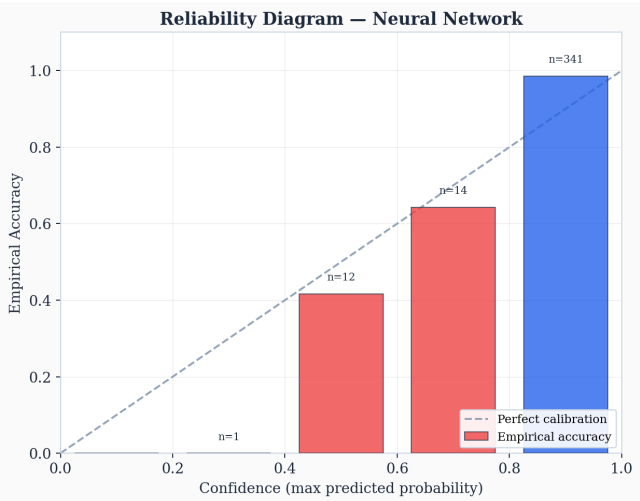

Figure 10: Reliability diagram for the neural network (data in Table 9). Nearly all predictions (92.7%) concentrate in the rightmost bin, where average confidence and average accuracy nearly coincide, showing excellent calibration.

Table 8: Reliability table — Softmax Regression (see Figure 9). Underconfidence is visible in the $[0.40, 0.60]$ and $[0.60, 0.80]$ bins where empirical accuracy greatly exceeds claimed confidence.

Bin	Count	Avg Conf	Avg Acc
$[0.00, 0.20]$	0	—	—
$[0.20, 0.40]$	13	0.338	0.308
$[0.40, 0.60]$	36	0.518	0.833
$[0.60, 0.80]$	48	0.708	0.875
$[0.80, 1.00]$	271	0.937	0.996

Table 9: Reliability table — Neural Network (see Figure 10). The NN concentrates 92.7% of predictions in the top-confidence bin, where average confidence (98.4%) and accuracy (98.5%) are nearly identical.

Bin	Count	Avg Conf	Avg Acc
$[0.00, 0.20]$	0	—	—
$[0.20, 0.40]$	1	0.360	0.000
$[0.40, 0.60]$	12	0.517	0.417
$[0.60, 0.80]$	14	0.724	0.643
$[0.80, 1.00]$	341	0.984	0.985

Interpretation. The neural network's tanh representation (Equation (13)) reshapes the input so that the output softmax (Equation (15)) becomes more decisive: most samples lie far from the decision boundary in hidden space, so probability mass on non-winning classes is very small. By contrast, the softmax model keeps many samples near an affine boundary (Equation (5)), which leads to mid-confidence predictions that are often better than their stated confidence. In short, the nonlinear model improves not only classification accuracy, but also the reliability of its probability estimates.

5.2 Repeated-seed evaluation

To measure the stability of our conclusions under random initialization, we trained both models on digits across 5 seeds and report 95% confidence intervals using the t-distribution ($t_{\text{crit}} = 2.776$, df = 4) [4].

Table 10: Repeated-seed evaluation (5 seeds, 95% CI) on digits. Non-overlapping cross-entropy intervals confirm that the NN is statistically superior in probability quality.

Model	Accuracy	Cross-Entropy
Softmax	0.9397 ± 0.0015	0.2694 ± 0.0001
Neural Network	0.9571 ± 0.0091	0.1566 ± 0.0162

The cross-entropy intervals do not overlap ($[0.2693, 0.2695]$ vs. $[0.1404, 0.1728]$), which gives strong statistical evidence that the NN is better calibrated in probabilistic terms. The softmax model shows almost no variance because its zero-initialization behaves nearly deterministically, while the NN shows a wider interval ($\pm 0.91\%$ accuracy) due to sensitivity to weight initialization.

6 Discussion

When is the linear model sufficient? On the linear Gaussian task, softmax regression matches the neural network exactly in both accuracy and cross-entropy (Table 1). The data geometry is affine-separable by derivation (Equation (31)), making additional nonlinear capacity superfluous.

What representational change does the hidden layer provide? On the moons task, the tanh hidden layer reshapes the 2D input into a representation where the interleaving crescents become linearly separable (Table 2 and Figure 4). The 12.5-ppt accuracy improvement is structural: no amount of training resolves the softmax model's limitation (Equation (5)).

When does complexity fail to justify itself? On the Gaussian task, the NN's additional 162 parameters over the softmax model's 6 provide no improvement (Table 1). On moons, increasing width from 8 to 32 also yields no gain (Table 4): the representational need saturates early, and over-parameterization is wasteful (though not harmful, due to L2 regularization).

What does the failure case teach? The width-2 failure (Table 6 and Figure 7) demonstrates that model capacity must match boundary complexity. Each tanh neuron contributes one smooth partition; two partitions cannot approximate the curved arc boundary. The model converges—but to the best solution within its limited function class, which is representationally incompatible with the task.

What do repeated-seed statistics add? The non-overlapping cross-entropy CIs in Table 10 provide stronger evidence of the NN's superiority than any single-run comparison could. The softmax model's tiny variance reflects zero-initialization determinism; the NN's larger interval reflects initialization sensitivity, highlighting the value of repeated evaluation.

7 Limitations

- **Model scope:** Only two model classes are compared. Multi-layer architectures, alternative activations (ReLU, GELU), and other families (SVMs, decision trees) are untested [3].
- **Optimizer defaults:** The optimizer study uses fixed defaults; each optimizer could perform better with tailored hyperparameters. The comparison reflects default usability, not ceiling performance.
- **Dataset scale:** All datasets are small by modern standards ($\leq$ 1074 training examples). Conclusions may not generalize to high-dimensional, large-sample regimes [4].
- **Single architecture:** Only tanh activations are tested. ReLU-based networks may exhibit different capacity thresholds [6].
- **Calibration analysis:** The reliability analysis uses 5 bins, providing a coarse view. Finer binning would be statistically unreliable with only 368 test examples.
- **Fixed seed count:** Five seeds provide rough uncertainty estimates but are insufficient for rigorous hypothesis testing. Confidence intervals should be interpreted as approximate [4].

8 Conclusion

Across theory and empirical results, one pattern remains clear. When the decision boundary is affine, as in the linear Gaussian task, softmax regression already captures the right geometry and performs on par with the neural model. When the boundary is intrinsically curved, as in moons, the hidden layer changes the representation enough to recover a large and reliable performance gain.

On digits, the neural network is not only more accurate but also meaningfully better as a probabilistic model: cross-entropy is lower, repeated-seed intervals remain favorable, and confidence quality is stronger. Track B reinforces this point directly: reliability analysis shows that the neural network places most predictions in a high-confidence, well-calibrated regime, while the linear model remains systematically under-confident in middle-confidence bins.

The practical conclusion is straightforward: start with the simplest model that matches the data geometry, and increase nonlinear capacity only when empirical and geometric evidence show that linear structure is the bottleneck.

Acknowledgments

We thank the instructors and reviewers of the Math4AI capstone at the National AI Center — AI Academy for guidance and feedback throughout this project.

References

[1] Marc Peter Deisenroth, A. Aldo Faisal, and Cheng Soon Ong. 2020. *Mathematics for Machine Learning.* Cambridge University Press.
[2] ETH Zurich Learning and Adaptive Systems Group. 2024. Introduction to Machine Learning Course Notes. Course notes.
[3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning.* MIT Press.
[4] Kevin P. Murphy. 2022. *Probabilistic Machine Learning: An Introduction.* MIT Press.
[5] Andrew Ng and Kian Katanforoosh. 2024. CS229 Lecture Notes: Deep Learning. Stanford University lecture notes.
[6] Stanford CS231n Staff. 2024. Neural Networks Part 1: Setting up the Architecture. Course notes.
[7] Stanford CS231n Staff. 2024. Optimization Part 2: Backpropagation, Intuitions. Course notes.

A All Experimental Figures

This appendix collects all generated plots at larger display sizes for detailed inspection. Each figure is accompanied by an extended explanation connecting the visual evidence to the mathematical framework developed in Sections 2.2, 2.4 and 2.7.

A.1 Linear Gaussian Task

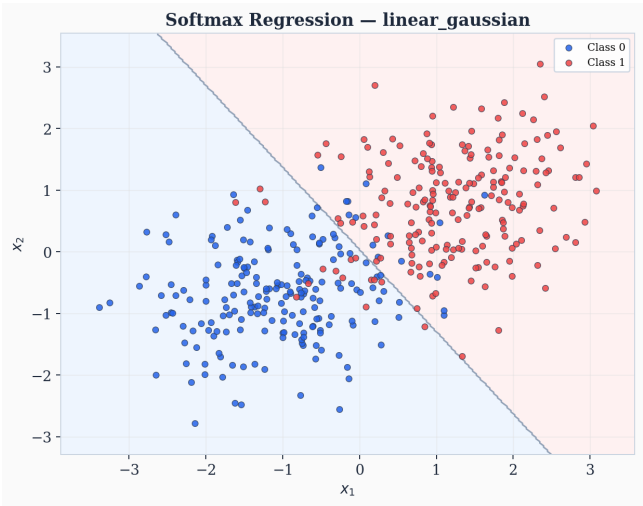

Figure 11: Softmax regression decision boundary — linear Gaussian. The shaded regions are the two predicted class regions; the gray line is the boundary where $p_0(\mathbf{x}) = p_1(\mathbf{x})$. As derived in Equation (31), this is an affine hyperplane with normal $\Sigma^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$. The clean separation of the two Gaussian clouds, with only a small overlap zone near the boundary, explains the 95.0% test accuracy in Table 1: the task is perfectly matched to the model's expressiveness. The slight curvature in the background shading is a rendering artifact of the mesh; the actual decision boundary is exactly straight.

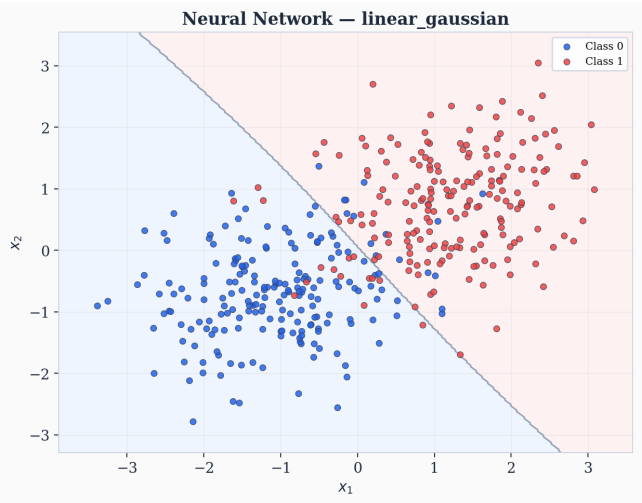

Figure 12: Neural network decision boundary — linear Gaussian. Despite having 162 parameters versus the softmax model's 6, the NN learns an approximately linear boundary virtually identical to Figure 11. This confirms the theoretical prediction of Section 2.7: when the Bayes-optimal boundary is affine (Equation (31)), the extra tanh hidden layer (Equation (13)) provides no geometric advantage. In practice, the tanh units learn to operate near zero pre-activation (the approximately linear regime of tanh), effectively collapsing the nonlinearity and converging to the same solution as the linear model.

Figure 13: Training and validation curves — softmax regression, linear Gaussian. *Left:* Cross-entropy loss (Equation (11)) decreases smoothly from ≈ 0.56 toward ≈ 0.21, with training and validation curves nearly coinciding throughout. This near-perfect overlap indicates minimal overfitting, expected from a 6-parameter linear model on a 240-example training set. *Right:* Accuracy plateaus near 91–92% on both splits, consistent with the test result in Table 1. The small persistent gap ($\approx 1\%$) reflects irreducible class overlap near the Gaussian boundary (Equation (31)).

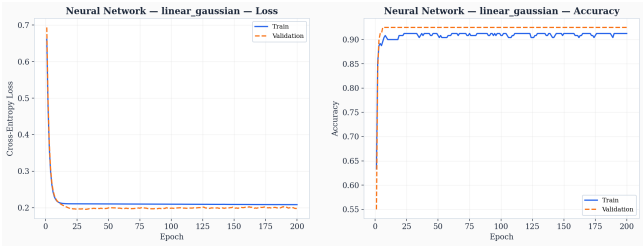

Figure 14: Training and validation curves — neural network, linear Gaussian. The loss curves follow the same qualitative profile as Figure 13, plateauing near 0.20. Adam's adaptive learning rates (Section 3.3) produce a faster initial drop than the softmax SGD, but both converge to the same loss floor, reflecting the identical representational capacity at this task. The accuracy panel shows more epoch-to-epoch noise (stochastic mini-batch ordering) and an unusual pattern where validation accuracy slightly exceeds training accuracy for much of training—a consequence of the small dataset size and the best-checkpoint restoration policy described in Section 3.3.

A.2 Moons Task

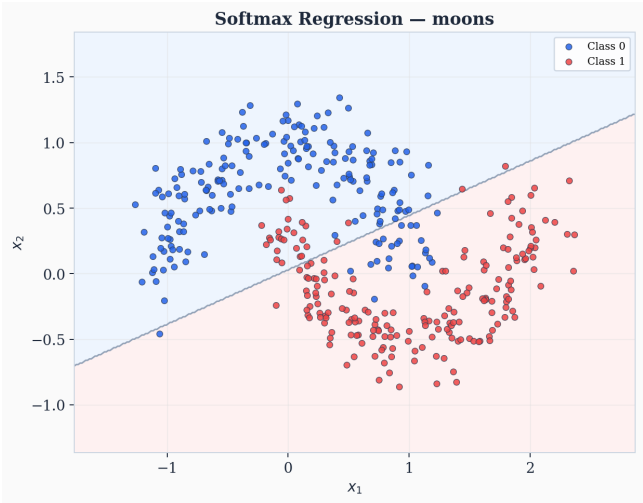

Figure 15: Softmax regression decision boundary — moons. The gray line is the single best affine separator that the softmax model class can produce. A wide arc of red (Class 1) points below and to the left of center, and a band of blue (Class 0) points in the upper-right, are systematically misclassified. The failure is *structural*: as proved in Equation (5), the boundary must be an affine hyperplane regardless of training duration, learning rate, or initialization. The model achieves 85.0% test accuracy (Table 2), the ceiling of the linear model class on this geometry.

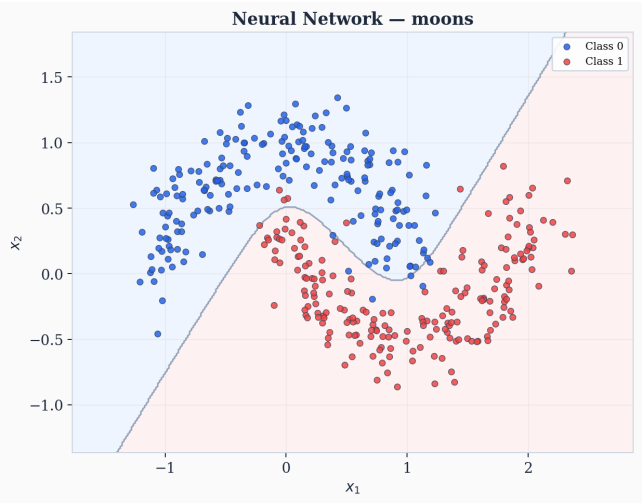

Figure 16: Neural network decision boundary — moons. The NN learns a smoothly curved, non-convex boundary that wraps around both crescents. Such shape is impossible for any affine model. The mechanism: the hidden layer (Equations (12) and (13)) maps the 2D input to a 32-dimensional feature space in which the two classes are approximately linearly separable; the output layer (Equation (14)) then places a hyperplane in that transformed space, which corresponds to this curved curve when projected back to the input domain. The result is 97.5% test accuracy (Table 2), a 12.5-ppt improvement over the softmax model in Figure 15.

Figure 17: Training and validation curves — softmax regression, moons. The validation loss (dashed) drops sharply in the first 25 epochs, then plateaus at ≈ 0.30. The early plateau is the visual signature of a *representational bottleneck*: the model has converged to the best linear fit, but further cross-entropy reduction would require a curved boundary that the linear model class cannot represent (Equation (5)). The accuracy (right panel) plateaus similarly near 86%. Notably, there is no overfitting (validation loss never rises), showing that the problem is expressiveness, not generalization. Compare to Figure 18, where the NN continues improving past epoch 25.

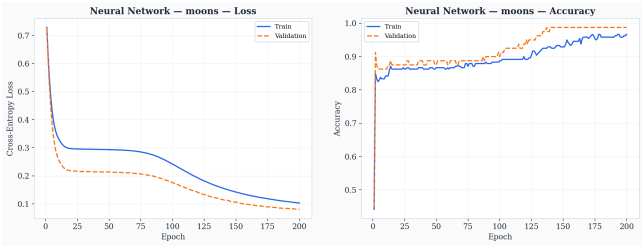

Figure 18: Training and validation curves — neural network, moons. Unlike Figure 17, the NN continues improving well past epoch 25. Validation loss (dashed) falls from ≈ 0.22 at epoch 25 to ≈ 0.09 at epoch 200; training loss (solid) follows more slowly, crossing the validation curve around epoch 125. The unusual training-above-validation behavior stems from the small training set (240 examples) and Adam's ability to rapidly exploit low-loss regions on the 80-example validation set. The final 97.5% test accuracy demonstrates representational learning, not mere memorization: the model generalizes well despite the training/validation loss gap.

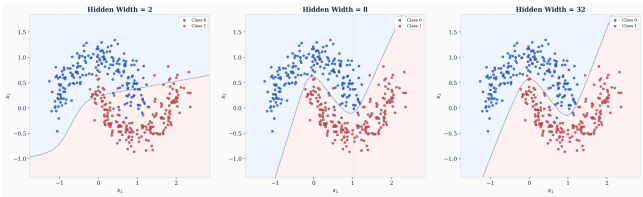

Figure 19: Capacity ablation — moons, hidden widths $\{2, 8, 32\}$. The three panels show how model capacity translates to decision boundary expressiveness. *Left (width 2):* Each tanh neuron produces one sigmoidal partition (Equation (13)); two partitions produce a nearly linear boundary, barely improving on the softmax model (86.25% accuracy, Table 4). *Center (width 8):* Eight neurons provide enough composable step-functions to form a curved boundary that encircles the red crescent, yielding 97.5% accuracy. *Right (width 32):* The boundary is qualitatively identical to width 8 with slightly smoother contours; accuracy is identical at 97.5%. Saturation at width 8 confirms that the moons task imposes a finite representational requirement, and increasing width beyond this threshold adds parameters without benefit.

A.3 Digits Benchmark

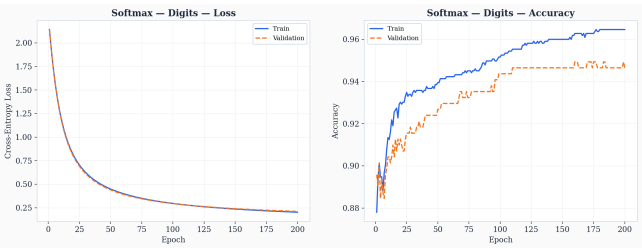

Figure 20: Training and validation curves — softmax regression, digits. *Left:* Both loss curves decrease from ≈ 2.10 (close to the random-initialization baseline $-\log(1/10) \approx 2.30$ for 10 classes) to convergence plateaus near 0.22 (train) and 0.27 (validation). The persistent train/val gap indicates mild overfitting despite only 6-parameter-per-class linear coefficients, suggesting the model begins fitting noise in the 1074-example training set. *Right:* The step-like accuracy curves result from the small validation set (355 examples), where each correctly classified example changes accuracy by $\approx 0.28\%$. Final test accuracy is 0.9402 (Table 3).

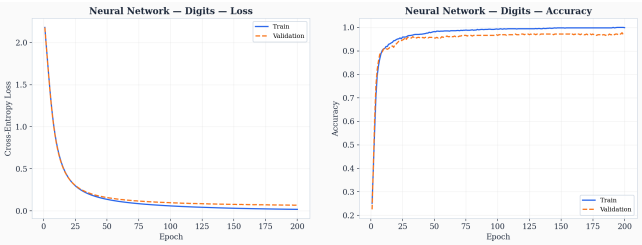

Figure 21: Training and validation curves — neural network, digits. The NN converges significantly faster and achieves far lower loss than the softmax model (Figure 20). Training cross-entropy (solid) approaches near zero by epoch 200, while validation cross-entropy (dashed) stabilizes near 0.07, less than a quarter of the softmax model's validation loss. The large train/validation gap points to overfitting on 1074 examples; the best-checkpoint policy (Section 3.3) restores the weights from epoch 198 (Table 3), where validation loss is minimized. The smoother accuracy curves compared to the softmax staircase reflect the NN's more confident predictions (see Table 7).

Figure 22: Optimizer comparison — digits, NN (h=32). Validation cross-entropy (left) and accuracy (right) over 200 epochs for SGD, Momentum, and Adam. *SGD (blue):* Slowest convergence; monotonically decreasing but well above the other optimizers through epoch 50. *Momentum (orange):* Fastest early convergence: the accumulated gradient velocity ($\mu = 0.9$) accelerates descent in consistent-gradient directions, reaching $\approx 93\%$ validation accuracy by epoch 10 while SGD is still at $\approx 90\%$. Momentum's final cross-entropy (0.1653) exceeds Adam's, possibly because the fixed step size overshoots sharp minima. *Adam (green):* Initially comparable to Momentum but achieves the lowest final cross-entropy (0.1507) by adapting per-parameter learning rates. All three converge to near-identical accuracy ceilings, verifying that the performance bound is set by model architecture, not optimizer. Numerical results in Table 5.

A.4 Track B: Reliability Diagrams

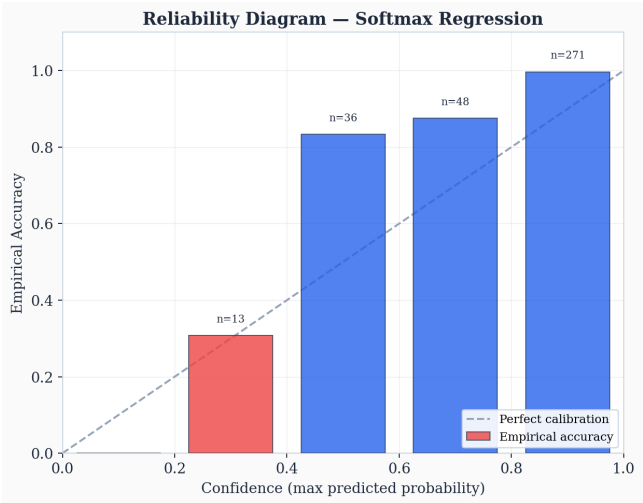

Figure 23: Reliability diagram — softmax regression, digits. Each bar corresponds to a 0.20-wide confidence bin; bar height is the empirical accuracy within that bin; red bars fall below the diagonal (under-confident) and blue bars above (over-confident). The softmax model is *systematically under-confident*: in the $[0.40, 0.60]$ bin (n=36), claimed confidence is $\approx 52\%$ yet accuracy is $\approx 83\%$; in the $[0.60, 0.80]$ bin (n=48), claimed confidence is $\approx 71\%$ yet accuracy is $\approx 88\%$. The under-confidence arises because the linear feature space (Equation (1)) places many examples near the affine decision boundary (Equation (5)), so softmax spreads probability mass across classes even when the correct class is apparent. The highest-confidence bin ($[0.80, 1.00]$, n=271) is near-perfectly calibrated. Data in Table 8.

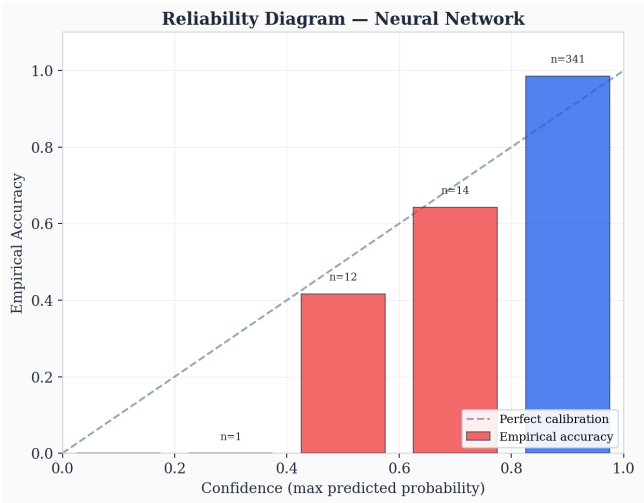

Figure 24: Reliability diagram — neural network, digits. Most notably, there is an extreme concentration of predictions: 341 of 368 test examples (92.7%) fall in the $[0.80, 1.00]$ bin, where empirical accuracy (98.5%) and mean confidence (98.4%) are virtually identical, showing near-perfect calibration. The two middle bins cover very few predictions (n=12 and n=14), and both are under-confident (red), consistent with the softmax model's pattern. The concentration arises because the tanh hidden layer (Equation (13)) reshapes features, placing most examples far from the output decision boundary in hidden space, and driving the softmax (Equation (15)) to assign near-zero probability mass to non-winning classes. Data in Table 9.

B Contribution Statement

- **Milana Karimova:** Built the training infrastructure: mini-batch loop with best-validation-checkpoint logic (`train.py`), SGD, Momentum, and Adam optimizers with bias correction (`optimizers.py`), and evaluation utilities with repeated-seed CI computation (`evaluate.py`). Ran the 5-seed repeated evaluation and authored the full Track B confidence and reliability analysis (`run_repeated_seeds.py`, `run_advanced_track.py`). Designed and produced the final presentation slide deck.
- **Fereh Feyzullayev:** Developed the softmax regression model with forward pass, L2-regularized loss, and gradient backpropagation (`softmax_regression.py`), along with all plotting utilities covering decision boundaries, training curves, and reliability diagrams (`visualize.py`). Ran the synthetic dataset experiments and generated all corresponding figures (`run_synthetic.py`). Compiled and submitted the final LaTeX report and slide deck, incorporating contributions from all team members.
- **Nijat Aghayev:** Wrote the shared math utilities, including numerically stable softmax, log-softmax, cross-entropy, and one-hot encoding (`math_utils.py`), as well as the data-loading module and mini-batch iterator (`data_loader.py`). Constructed the one-hidden-layer neural network with Xavier initialization, tanh activation, and vectorized backpropagation (`neural_network.py`). Ran the digits benchmark and all ablation experiments, and compiled the numeric results summary (`run_digits.py`, `run_ablations.py`, `results/README.md`). Co-authored the report.